

1972 CitiCar EV Restoration and Automation



Project Overview

This document tracks the current progress of the Autonomous Self-Driving 1972 CitiCar a personal restoration and modernization project begun in March 2025 and still ongoing. The objective is to restore the chassis, body, and mechanical systems while modernizing the powertrain and electronics to enable autonomous driving and control. The sections below summarize the state of each subsystem: restoration, motor, controller, and software.

Restoration

Restoration work covers the chassis, body, and mechanical systems of the original 1972 vehicle, alongside a full modernization of the power system.

- Chassis, body, and mechanical systems: being restored to serviceable condition as the foundation for the modernized powertrain and electronics.
- Battery pack: a new 48V pack was built from two Tesla Model S modules, providing an estimated 50-mile range and replacing the original lead-acid system.

Motor

The original drive motor has been restored and retained rather than replaced, preserving the vehicle's character while feeding a modernized control system.

- Restored the original 3.5 hp, 48V DC motor to working condition.
- Paired the motor with a Curtis 1205M motor controller, matching the new 48V pack voltage.

Controller

The powertrain uses a Curtis 1205M-5603 motor controller, normally configured and monitored with the Curtis 1314 PC Programming Station. To integrate the controller into the car's autonomous control system, its telemetry interface was reverse engineered so live data can be read and republished onto the vehicle CAN bus.

- Reverse-engineered the Curtis 1205M controller's RS232 serial telemetry protocol used by the Curtis 1314 PC Programming Station.
- A microcontroller reads live telemetry from the controller and translates it into CANopen frames, integrating the data into the car's CAN bus.
- All controller interaction is read-only the tooling only sends handshake and read frames that have been explicitly captured and verified.

Software

The Curtis 1205M controller was designed to be monitored only through Curtis's proprietary Windows programming software, which doesn't expose its data to anything else. To make the controller's telemetry available to the car's autonomous systems, I wrote my own set of read-only tools to capture that communication, understand it, and re-broadcast it in a form the rest of the vehicle can use.

Capturing and Decoding the Protocol

The first step was learning how the controller and the Curtis software talk to each other. I built tooling that records the serial conversation in both directions so it can be studied. Because the Curtis software and my tools can't share the same serial port at once, the capture is done through a virtual bridge — and, when that caused the software to time out, by passively sniffing the wires or hooking the Curtis program directly. The recorded traffic is then grouped into request and reply to pairs to work out which messages are routine telemetry and what the controller's responses mean. This is how the RS232 protocol was reverse engineered.

Live Monitoring and CAN Integration

Once the protocol was understood, I built a live dashboard that displays the controller's telemetry in real time, and microcontroller firmware that continuously polls the controller and republishes the data as CANopen frames onto the car's CAN bus. This puts the motor

controller's live data on the same network as the rest of the vehicle's electronics, where the autonomous control system can use it.

Safety

Everything is read-only by design. The tools only send messages that have been explicitly captured and verified, so they never issue unknown commands to the controller, and the default configuration sends nothing at all until a command has been confirmed safe.

Automation

With the powertrain and telemetry in place, the next phase adds the actuation, sensing, and control needed for autonomous driving. The car's existing mechanical controls will be adapted for computer actuation, and an established open-source autopilot stack will handle navigation and path planning.

- Steering: a DIY servo-controlled pinion will be coupled to the steering wheel shaft, giving the computer direct control of the car's steering.
- Braking: a CAN-controlled brake master cylinder will provide computer-actuated braking, commanded over the car's CAN bus.
- Environmental sensing: a LiDAR sensor will provide environmental awareness for obstacle detection and mapping of the car's surroundings.
- Localization: an IMU and GPS will track the vehicle's position, orientation, and motion.
- Path planning: these sensors and actuators will feed ArduRover, which will handle automatic path planning and autonomous navigation.